TAB C — Load Testing

C.1 What the Government asked for, restated as design constraints

TE3 and the Q&A answers give six constraints, and each one narrows the design:

1. **Any developer must be able to run the tests locally**, even where local numbers do not reflect production (TE3; Q&A 38) — so the harness cannot depend on cloud infrastructure to exist at all.
2. **The same tool must scale to an environment of "more verisimilitude"** as an option, not a rewrite (Q&A 38) — HEC's Apache CloudStack environment at no cost, or another if demonstrably better (Q&A 39).
3. **Open-source, unrestricted licensing** (Q&A 38).
4. **The target of interest is CDA response time**, regardless of the calling application, so that caching strategies can be analysed (Q&A 38); the stated intent is to learn whether RDS needs vertical scaling or read replicas and whether CDA benefits from horizontal scaling (TE3).
5. **One, two, and three CDA instances** (round-robin) must each be testable, but as a configuration property rather than a mandatory triple run (Q&A 40); CPU/memory fixed at 1 vCPU / 2 GB per CDA instance (TE3).
6. **This is baseline characterization, not performance targets** (TE3; Q&A 40): the five load rows are controlled attempts, coverage of read/write and authenticated/unauthenticated combinations should be "as many as practical" starting from a reasonable sample, and the minimum facilities are authorization, time-series storage and retrieval with catalogs, and location storage and retrieval with catalogs.

C.2 Tool: k6, and why

TE3 names Apache JMeter as an example; the previous effort's design document listed JMeter but its repository actually contains **k6** scripts (`tools/benchmark/scenarios.js`, `quick-benchmark.js`, `run-benchmark.sh`) with Keycloak password-grant tokens for the persona test users and thresholds on p95/p99 latency. We propose to keep k6 and build the harness on it, for four reasons that map to the constraints above:

- **Local by default.** k6 is a single static binary; a scenario is a JavaScript file; a run is one command against `http://localhost:8081/cwms-data`. Nothing about it assumes a cluster.
- **Scales without changing the artifact.** The same script runs distributed (k6 in Docker across CloudStack VMs, or the open-source `k6-operator` on Kubernetes) — the "option within the same tool" the Government described.
- **Open source, Apache 2.0**, with no GUI-bound test definitions to version-control.
- **It is already there.** Continuity with the existing scripts, tokens, and report format (`docs/benchmarks/`), which HEC has seen.

JMeter remains a legitimate alternative and we would switch if HEC prefers it: the scenario matrix in C.4 is tool-neutral and the harness layout keeps the runner behind one script so that a JMeter `.jmx` set could replace the k6 files without touching the rest. What the previous scripts

lack — and what Task 4 adds — is the multi-instance topology, the write paths, the data-seeding step, and the result analysis.

C.3 Harness layout

Everything lives in the cwms-data-api repository under `load-tests/` so that the code under test and the tests that load it are versioned together and the harness is maintained under Task 5 like any other code.

```
load-tests/
  README.md                 how to run locally, against CloudStack, against CWBI-Dev
(read-only)
  config/
    local.json              target base URL, instance count, auth mode, data set,
durations
    cloudstack.json
    cwbi-dev.json           read-only scenarios only; never write to a shared
environment
  scenarios/
    auth.js                 authorization mechanisms: unauthenticated, API key, OIDC
bearer, via-proxy vs direct
    timeseries.js           GET /timeseries (several window lengths), POST
/timeseries, catalog TIMESERIES
    locations.js            GET /locations, GET /locations/{id}, catalog LOCATIONS,
POST /locations
    ratings.js              POST /ratings/rate-values and /rate-ts (the "rate"
endpoint)
    mix.js                  read-heavy composite (the Government's stated assumption)
  lib/
    tokens.js               Keycloak password grant for persona users; API-key header
helper
    data.js                 identifier lists generated from the seeded data set
    windows.js              begin/end generators for 1 h, 1 d, 7 d, 30 d, 365 d
windows
  seed/
    seed.py                 loads the load_data/ CSV and parquet fixtures (LRL, MVP)
into the target DB
  topology/
    docker-compose.load.yml overlay: `--scale data-api=N` behind Traefik round-robin;
OPA/proxy/cache profiles
  run.sh                    one entry point: run.sh <config> <scenario> [--instances
N] [--auth mode] [--profile proxy]
  results/                  JSON summaries per run; analysis notebook reads them
  analysis/
    baseline.ipynb          p50/p95/p99, throughput, error rate by scenario ×
instances × auth × window
```

Targeting a specific system (TE3: "reasonably easy for a developer to target a specific system") is the `config/*.json` file plus `--instances`; nothing else changes between a laptop and CloudStack.

Instance topology. CDA's compose file already carries the Traefik load-balancer label (`traefik.http.services.data-api.loadbalancer.server.port=7000`); the load overlay adds `--scale data-api=N` and a health-checked round-robin so that one, two, or three instances are a number in the config, exactly as Q&A 40 asked. Each instance is pinned at 1 vCPU / 2 GB (TE3) through compose resource limits, and the same pins carry to CloudStack VMs. The authorization profile adds the proxy, OPA, and cache containers in front so that "via proxy" and

"direct to CDA" can be compared on identical runs — the difference is the authorization overhead the design is meant to keep under a few milliseconds.

Data. The repository's `load_data/` notebooks and the LRL/MVP location and melted-time-series fixtures give a realistic seed; `seed.py` loads them once per environment and generates the identifier lists the scenarios draw from, so every developer runs against the same data shape. Write scenarios use a dedicated office and identifier prefix and clean up after themselves; the `cwbi-dev.json` config disables writes outright.

C.4 Scenario matrix

Table C-1. Load rows from TE3 × topology × mode. Every cell is a config combination, not a separate script; the run script iterates the ones selected.

TE3 load row	Users / rate / parallelism	Instances	Auth modes	Read/write mix
L1	1 user, 50 req/s, parallelism 10	1, 2, 3	unauth, API key, OIDC, via-proxy	read; write
L2	100 users, 50 req/s each, parallelism 10	1, 2, 3	unauth, OIDC, via-proxy	read-heavy mix (90/10)
L3	1,000 users, 100 req/s aggregate	1, 2, 3	OIDC, via-proxy	read-heavy mix
L4	10,000 req/s (controlled attempt)	1, 2, 3	unauth, via-proxy	read
L5	100,000 req/s (controlled attempt)	3	unauth	read (catalog + timeseries)

Per Q&A 40, L1–L3 are interpreted as a combination of parallel and sequential virtual users; L4 and L5 are attempts whose purpose is to find where the stack saturates (CDA CPU, connection pool, RDS I/O, or the load generator itself — the harness records which). Each scenario runs the facilities TE3 lists as minimum: authorization (token acquisition and a decision-bearing request), time-series `GET` across five window lengths and one `POST`, catalog of time series, location `GET`/`POST` and catalog of locations, and the rating endpoints. Coverage of the remaining endpoints is added as the sample grows; TE3 accepts a single `POST` and `GET` per mechanism for this baseline and does not require 100 % (Q&A 40).

Runs per cell are short (60–120 s steady state after a 30 s ramp) so that a full local sweep of the read cells fits in an evening on a laptop, and a CloudStack sweep including L4/L5 fits in a day.

C.5 What the baseline answers, and what we deliver

The Government named three questions and two observations, and the analysis notebook is organized around them:

- **RDS vertical vs. read replica.** Compare RDS-side latency and I/O across L1–L3 with one CDA instance (isolating the database) against two and three instances (adding API concurrency); a database-bound curve flattens as instances rise.
- **CDA horizontal scaling.** Throughput and p95 per added instance at fixed load; the point where a third instance stops helping is the database or the proxy, and the via-proxy/direct comparison says which.

- **Caching strategy.** Via-proxy runs with a cold and a warm decision cache show the authorization overhead and the cache hit rate; the previous effort's single-instance measurement (about 2–3 ms proxy overhead, 99.7 % Redis hit rate on a developer machine) becomes a multi-instance figure on shared infrastructure.
- **"Write amplification" and bot downloads (Q&A 38).** Two dedicated scenarios: hourly-style write bursts (GOES-shaped, small and frequent) followed by computations, to reproduce the block-storage throttling HEC has observed; and a small number of clients pulling large time-series windows continuously, to characterize what a download bot costs the rest of the system. Both feed the caching discussion — a read-through cache in front of catalog and long-window reads is the obvious candidate and the numbers will say whether it is worth it.

Deliverables (Task 4): the harness and overlay in the repository with a README that a new developer can follow in under an hour; seeded data and identifier lists; a results folder with the JSON summaries of the baseline sweep on the local stack and on CloudStack; the analysis notebook and a short baseline report (tables of p50/p95/p99, throughput, error rate, and saturation point per cell, with the three questions answered as far as the data allows and the next experiments suggested); and a GitHub Actions job that runs a smoke subset of the read scenarios on every pull request to `develop` so that the harness does not rot between uses. No performance targets are asserted — TE3 is explicit that there are none — but the harness is built so that HEC can set them later and let CI enforce them.